
GRADO EN INGENIERÍA EN SISTEMAS DE
INFORMACIÓN 2025/2026

Trabajo Fin de Grado

Evaluación del potencial de los LLMs en tareas de ofuscación y desofuscación de código

AUTOR

Gabriel López Ormeño

TUTOR

Alberto Ballesteros Rodríguez

Alcalá de Henares, junio 2026

Resumen

Las tareas de ofuscación y desofuscación de código son procedimientos ampliamente conocidos en el mundo del desarrollo del software. Desde sus inicios, estas se han llevado a cabo mediante mecanismos deterministas fundamentados en las alteraciones sintácticas y la heurística.

No obstante, en los últimos años, el auge de los modelos de lenguaje a gran escala (LLMs) ha posibilitado la opción de que estos sean utilizados en este tipo de procesos, al ser capaces de comprender y generar código.

En el presente trabajo se tratará de evaluar el rendimiento de distintos LLMs en la ofuscación y desofuscación de código JavaScript. Para ello, se ha desarrollado desde cero un framework en Python, haciendo uso de las herramientas de Ollama y LangChain. Este sistema permite ejecutar experimentos, seleccionando el modelo con el que se quiere interactuar, el script a transformar, la estrategia y configurando su prompt.

Los modelos que se han decidido evaluar son llama3.2:3b, llama3.1:8b, qwen2.5-coder:7b y mistral:7b, incluyendo así tanto modelos de propósito general, como de especialización en software.

La evaluación se ha desempeñado principalmente en dos direcciones: la similitud estructural del código generado con respecto al original y su funcionalidad, observando los resultados de la ejecución.

Todo ello permite establecer una comparativa entre los distintos modelos, identificando sus características frente a los diferentes datasets y estrategias, para poder así extraer conclusiones sobre las ventajas y limitaciones de los LLMs frente a los métodos tradicionales.

Abstract

Code obfuscation and deobfuscation are well-established practices in the world of software development. Since their inception, these processes have been carried out using deterministic mechanisms based on syntactic alterations and heuristics.

However, in recent years, the rise of large-scale language models (LLMs) has made it possible to use them in these types of processes, as they can understand and generate code.

This paper aims to evaluate the performance of different LLMs in the obfuscation and deobfuscation of JavaScript code. To this end, a Python framework was developed from scratch, utilizing tools from Ollama and LangChain. This system allows users to run experiments by selecting the model to interact with, the script to transform, the strategy, and configuring the prompt.

The models selected for evaluation are llama3.2:3b, llama3.1:8b, qwen2.5-coder:7b, and mistral:7b, thus including both general-purpose models and those specialized in software.

The evaluation was conducted primarily in two areas: the structural similarity of the generated code to the original and its functionality, based on the execution results.

This allows us to compare the different models, identifying their characteristics across various datasets and strategies, and thereby draw conclusions about the advantages and limitations of LLMs compared to traditional methods.

Índice general

Resumen.....	2
Abstract.....	3
1. Introducción.....	7
1.1. Introducción.....	7
1.2. Motivación.....	8
2. Objetivos y alcance.....	9
2.1. Objetivo principal.....	9
2.2. Objetivos específicos.....	9
2.3. Alcance del proyecto.....	9
3. Metodología.....	10
3.1. Estrategia metodológica.....	10
3.2. Fases.....	10
3.3. Herramientas y tecnologías.....	11
4. Marco conceptual.....	12
4.1. Ofuscación de código.....	12
4.1.1. Definición y propósito.....	12
4.1.2. Técnicas de ofuscación.....	12
4.2. Desofuscación de código.....	13
4.2.1. Definición y enfoques tradicionales.....	13
4.2.2. Técnicas de desofuscación.....	13
4.3. Modelos de lenguaje a gran escala (LLMs).....	14
4.3.1. Arquitectura general.....	14
4.3.2. Aplicaciones en ingeniería del software.....	15
4.4. Prompt engineering.....	16
4.4.1. Definición y relevancia.....	16
4.4.2. Estrategias principales.....	16
5. Estado del arte.....	17
5.1. Capacidades de los LLMs en tareas de transformación de código.....	17
5.1.1. Compresión semántica del código por parte de los LLMs.....	17
5.1.2. LLMs aplicados a la desofuscación de código.....	17
5.1.3. LLMs como generadores de código ofuscado.....	18
5.2. Síntesis y posicionamiento del trabajo.....	19
6. Desarrollo del framework.....	20
6.1. Arquitectura del sistema.....	20
6.2. Estructura del repositorio.....	21

6.3.	Flujo de ejecución	22
6.3.1.	Selección del experimento	22
6.3.2.	Generación de logs	22
6.3.2.	Evaluación de resultados	23
6.3.4.	Representación de gráficas	23
7.	Experimentación	24
7.1.	Diseño experimental	24
7.2.	Modelos evaluados	24
7.3.	Definición de datasets	25
7.4.	Configuración experimental	26
7.3.1.	Diseño de prompts	26
7.3.2.	Parámetros LLM	28
8.	Métricas y evaluación	30
8.1.	Métricas	30
8.1.1.	Similitud estructural: AST score y node ratio	30
8.1.1.	Funcionalidad: error rate y functional match	31
8.1.3.	Rendimiento: tiempo de ejecución	32
8.1.4.	Línea de base: ofuscación tradicional	32
8.2.	Evaluación de resultados	33
8.2.1.	Similitud estructural y calidad del código generado	33
8.2.2.	Resultados por tarea: ofuscación y desofuscación	35
8.2.3.	Impacto de la estrategia de prompting	37
8.2.4.	Tiempos de ejecución	39
9.	Conclusiones y reflexión	41
9.1.	Conclusiones	41
9.2.	Líneas futuras	43
9.3.	Reflexión final	44
10.	Bibliografía	45

Índice de figuras

Tabla 1. Herramientas y librerías del framework	11
Tabla 2. Estructura del repositorio	21
Tabla 3. LLMs evaluados	25
Tabla 4. Tabla comparativa de ofuscación	34
Tabla 5. Tabla comparativa de éxito funcional por tarea	36
Tabla 6. Tabla comparativa de éxito funcional por estrategia.....	38
Tabla 7. Tabla comparativa de tiempo medio de ejecución.....	40
Ilustración 1. AST Score en la ofuscación.....	33
Ilustración 2. Node ratio en la ofuscación.....	33
Ilustración 3. Functional match en la ofuscación	34
Ilustración 4. Tasa de error por estrategia.....	35
Ilustración 5. Éxito funcional por estrategia.....	36
Ilustración 6. Éxito funcional por estrategia	37
Ilustración 7. Tasa de error por estrategia.....	38
Ilustración 8. Tiempo medio por modelo y tarea.....	39
Ilustración 9. Mapa tiempo de ejecución/ast score	39

1. Introducción

1.1. Introducción

La ofuscación y la desofuscación de código son técnicas consolidadas en los campos de la seguridad y desarrollo software.

La primera mencionada convierte un programa en una versión de sí misma capaz de realizar lo mismo funcionalmente, pero con una mayor dificultad a la hora de ser comprendida. Sus motivos de uso son varios, desde la protección de la propiedad como la evasión de ciertos mecanismos de análisis y detección software. La segunda persigue el objetivo contrario, es decir, a partir de un código ya ofuscado e ilegible lograr obtener su versión comprensible y estructurada conservando también su funcionalidad. La desofuscación se utiliza especialmente en tareas como la seguridad, el análisis de malware o el mantenimiento de software.

Históricamente, ambas tareas han sido abordadas mediante métodos tradicionales: transformaciones sintácticas y estructurales automatizadas en el caso de la ofuscación, y técnicas heurísticas basadas en descompiladores y reconocimiento de patrones en el de la desofuscación. Estos enfoques demuestran una gran eficiencia, pero son predecibles y están sujetos a limitaciones propias de su naturaleza metódica y automatizada, ya que operan sobre la estructura del código sin capacidad real de razonar sobre su semántica.

En este contexto, la irrupción de los LLMs abre una nueva posibilidad, al ser entrenados sobre cantidades masivas de código fuente en múltiples lenguajes de programación y haber demostrado una notable capacidad para comprender y generar código. Esta competencia sugiere que podrían, en principio, abordar las tareas de ofuscación y desofuscación desde una perspectiva semántica, razonando sobre el propósito del código y no únicamente sobre su forma sintáctica.

Sin embargo, la pregunta de si los LLMs son capaces realmente de desempeñar estos procesos de manera fiable, eficiente y comparable a los métodos tradicionales no está cerrada y es amplia por todas las posibilidades de experimentación existentes. Se encuentran estudios que apuntan tanto a resultados prometedores como a limitaciones importantes, pero la información disponible es aún escasa, fragmentada y difícilmente comparable entre sí.

El presente trabajo responde a esta pregunta de manera sistemática y controlada. Se ha desarrollado un framework en Python que permite configurar y ejecutar experimentos de ofuscación y desofuscación sobre un dataset de scripts escritos en el lenguaje de programación JavaScript, evaluando el rendimiento de cuatro modelos con los que se ha decidido interactuar. Existiendo dos estrategias de prompting distintas: zero-shot y few-shot. Los resultados se comparan en todo momento con los métodos tradicionales de ofuscación como línea de base, empleando métricas de similitud estructural y funcionalidad mediante ejecución real del código generado.

A lo largo de esta memoria se detalla el proceso completo seguido, desde la revisión bibliográfica y el diseño del framework, pasando por la construcción del dataset y la configuración experimental, hasta el análisis de los resultados obtenidos y las conclusiones extraídas sobre las posibilidades y limitaciones reales de los LLMs en este dominio.

1.2. Motivación

Este trabajo fue propuesto por Alberto Ballesteros Rodríguez como una de las líneas de trabajo disponibles para el alumnado, y entre todas las opciones a disposición fue la que más despertó mi interés desde el primer momento. La razón principal fue la combinación de la inteligencia artificial aplicada a código y la seguridad del software, dos áreas especialmente interesantes tratadas en la carrera.

Durante el grado tuve la oportunidad de cursar asignaturas con relación a la programación, como Programación o Ingeniería del Software; al igual que con relación a la seguridad informática, como Seguridad y Seguridad en Sistemas Distribuidos. Ese contacto previo con el ámbito de la seguridad hizo que la propuesta del tutor resonara de inmediato y la seleccionase en mi solicitud de temas como la primera.

Lo que terminó de inclinarme por este TFG frente al resto de opciones fue precisamente la naturaleza abierta de la pregunta que plantea. No se trata de implementar una solución a un problema conocido, sino de evaluar de forma rigurosa hasta dónde llegan realmente los LLMs, comparándolos con los métodos establecidos. Esa incertidumbre inicial y extensas posibilidades, lejos de ser un inconveniente, me pareció la parte más interesante y enriquecedora del proyecto, los resultados no estaban garantizados de antemano.

Existe además una motivación de carácter práctico, el malware recurre sistemáticamente a la ofuscación para evadir la detección, y los analistas de seguridad necesitan herramientas de desofuscación fiables para revertir ese proceso. Conocer en qué condiciones los LLMs pueden automatizar alguna de estas tareas es una información de valor real, no solo académico, sino de uso personal.

2. Objetivos y alcance

2.1. Objetivo principal

El objetivo principal de este proyecto es la evaluación de los LLMs de manera controlada y sistemática en las tareas de ofuscación y desofuscación de código. Comparando sus resultados con la ofuscación tradicional y estableciendo un marco para lograr conocer en detalle las ventajas y limitaciones de los modelos a evaluar.

2.2. Objetivos específicos

Para lograr este objetivo principal, es necesarios superar otros objetivos más específicos:

- Diseñar e desarrollar un framework que permita ejecutar experimentos de ofuscación y desofuscación sobre datasets predefinidos, de manera automatizada.
- Construir datasets que incluyan scripts de tamaño y complejidad variable, con sus scripts correspondientes a cada uno ofuscados a través de los métodos tradicionales.
- Analizar el comportamiento de los modelos bajo escenarios zero-shot y few-shot, indagando en el impacto de la estrategia de prompting en los resultados obtenidos
- Diseñar e implementar un módulo de evaluación que mida tanto la similitud estructural del código generado como su correctitud funcional.
- Comparar el rendimiento de los LLMs frente a los métodos tradicionales de ofuscación, identificando patrones de comportamiento en función del tipo de modelo, el tamaño del código y la estrategia.
- Extraer conclusiones sobre las posibilidades y limitaciones de los LLMs en este dominio.

2.3. Alcance del proyecto

Los modelos evaluados son abiertos y ejecutables de manera local a través de Ollama, esto permite que quien quiera pueda replicar un escenario concreto y trabajar con el framework y sus modelos. Además, con esto logramos no depender de servicios de pago y tener más privacidad al no exponer la información a internet.

Los modelos que específicamente se han seleccionado no superan los 8 billones de parámetros. Esta decisión responde a un enfoque orientado a la ejecución en hardware de uso personal, sin necesidad de infraestructuras especializadas como GPUs de alto rendimiento o entornos distribuidos. De este modo, se garantiza la reproducibilidad de los experimentos en contextos accesibles, alineando el proyecto con escenarios realistas tanto académicos como profesionales.

En este trabajo no se aborda la ofuscación a nivel compilado o bytecode, ni tampoco la generación de malware, se enfoca exclusivamente en el código de alto nivel en texto plano.

3. Metodología

3.1. Estrategia metodológica

La metodología empleada es experimental de carácter iterativo. Con este enfoque se ha ido analizando el comportamiento de los LLMs en diferentes entornos, siguiendo los resultados con la evolución y variación de distintos módulos y configuraciones.

La metodología se fundamenta en la integración y coherencia de tres partes: el entorno experimental, los datos de entrada y el sistema. Estos elementos se combinan dentro del framework desarrollado, lo que permite la ejecución sistemática de experimentos y la obtención de resultados consistentes.

El enfoque iterativo ha permitido refinar progresivamente tanto la estructura como la funcionalidad del sistema, incorporando mejoras cada una de las partes, en función de los resultados obtenidos durante el desarrollo.

3.2. Fases

El desarrollo del proyecto se ha organizado en las siguientes fases:

1. Revisión bibliográfica y estado del arte: búsqueda y análisis de literatura académica sobre LLMs, ofuscación y desofuscación de código. Conocimiento de las principales librerías y frameworks disponibles.
2. Selección y análisis de herramientas: pruebas de adecuación y selección de las herramientas para el entorno del proyecto.
3. Construcción del dataset: creación de scripts manuales de distinto propósito y complejidad. Obtención de sus versiones ofuscadas a través de la ofuscación tradicional.
4. Desarrollo del framework: implementación del sistema de ejecución de experimentos y del módulo de evaluación. Diseño de los prompts y ajuste de las configuraciones de los modelos.
5. Ejecución de experimentos: lanzamiento sistemático de los distintos experimentos sobre el dataset completo, cubriendo todos los modelos, escenarios y tareas definidos.
6. Análisis de resultados: procesamiento de los resultados obtenidos, cálculo de métricas, generación de gráficas comparativas y extracción de conclusiones.

3.3. Herramientas y tecnologías

A continuación, van a ser detalladas a modo de tabla las diferentes herramientas y sus versiones exactas que han sido utilizadas en el framework desarrollado:

Herramienta o librería	Versión	Propósito
Python	v3.10+	Lenguaje de programación principal del software
Node.js	v18+	Entorno de ejecución para herramientas y scripts en JavaScript
Ollama	v0.13.1	Ejecución local de modelos LLM
LangChain	langchain-ollama v1.0.1 langchain-core v1.2.5	Integración y configuración de los modelos
JS Obfuscator	v4.0.2	Ofuscación de código JavaScript
Esprima	v4.0.1	Análisis sintáctico de código JavaScript
Pandas	v0.0.3	Manipulación, análisis y estructuración de datos
Rich	v13.5.0	UI por consola mediante formateo avanzado y visualización

Tabla 1. Herramientas y librerías del framework

4. Marco conceptual

4.1. Ofuscación de código

4.1.1. Definición y propósito

La ofuscación de código puede ser definida como el conjunto de transformaciones que se llevan a cabo sobre un código con los que se busca complicar el entendimiento de este o su análisis por parte de otra persona o agente externo. El resultado de este procedimiento es un programa que devolverá el mismo resultado que el original pero semánticamente incomprensible.ⁱ

En el mundo del desarrollo, los motivos para querer realizar la ofuscación son diversos.

- Proteger la propiedad intelectual de un software
- Dificultar la ingeniería inversa
- Proteger secretos algorítmicos
- Reducir la superficie de ataque de un sistema.

En contextos maliciosos, el malware recurre sistemáticamente a la ofuscación para evadir la detección por parte de antivirus u otros actores de defensa.

4.1.2. Técnicas de ofuscación

Las técnicas que posibilitan la obtención del código ofuscado son las siguientes:ⁱⁱ

- Renombrado de identificadores: sustitución de los nombres de las variables, funciones y estructuras con términos sin significado alguno (por ejemplo, en ocasiones caracteres Unicode). Esta es la técnica más evidente y conocida, pero a la vez básica, conocida comúnmente.
- Inserción de código muerto: adición de código que no altera la lógica del programa o que directamente nunca se ejecutan, aumentando así la complejidad al menos visualmente del código y dificultando su análisis.
- Opacificación del flujo de control: transformación de las estructuras de control en formas equivalentes, pero más complejas, como el uso de predicados opacos o la introducción de saltos artificiales.
- Codificación de cadenas: codificación en distintos esquemas (el más conocido, Base64) de las cadenas que hay presentes en el código, que durante la ejecución son descifradas.
- Compactación y eliminación de espacio en blanco: eliminación de comentarios, espacios y saltos de línea, así como compresión del código en una única línea.

4.2. Desofuscación de código

4.2.1. Definición y enfoques tradicionales

La desofuscación de código se ocupa de recuperar el código original a partir de su versión ofuscada, constituyendo así el proceso inverso a la ofuscación. El objetivo ideal sería restituir el programa en su estado original, pero en la práctica esto resulta complejo o directamente irrealizable; en su lugar se persigue obtener una versión funcional e interpretable que permita comprender la lógica subyacente del programa.

Un resultado de desofuscación exitoso es aquel que, preservando el comportamiento semántico del código, devuelve a este su legibilidad y estructura comprensible para un analista humano.

Los propósitos para desofuscar código son igualmente variados.

- Revisión de binarios
- Análisis de malware
- Ingeniería inversa

4.2.2. Técnicas de desofuscación

Las técnicas de desofuscación se aplican de forma complementaria a las de ofuscación, buscando revertir o neutralizar cada una de las transformaciones que esta introduce. Las principales son las siguientes:

- Renombrado de identificadores: sustitución de los nombres de las variables o funciones, pero esta vez tratando de dar un nombre descriptivo en relación con el contenido o comportamiento de esta.
- Eliminación de código muerto: identificación de sentencias y líneas de código de inutilidad o relleno, para posteriormente, en la mayoría de los casos eliminarlas.
- Simplificación del flujo de control: reestructuración de estructuras complejas de código para facilitar su comprensión, como saltos de líneas generados y otras características de la ofuscación.
- Decodificación de cadenas: se identifican las rutinas de descifrado presentes en el código y se aplican para recuperar las cadenas originales, ya sea mediante ejecución parcial del código o mediante análisis estático de los esquemas de codificación empleados.
- Reformateo y reconstrucción estructural: se restituyen los espacios en blanco, saltos de línea e indentación eliminados durante la compactación, y se reconstruye la jerarquía lógica del código para devolver al programa una organización legible.

Para determinar si una desofuscación ha sido exitosa, la literatura emplea métricas de naturaleza tanto estructural como funcional.

En cuanto a la estructura, se puede observar como la más utilizada en este ámbito es el Abstract Syntax Tree score (AST score), que compara la sintaxis del código generado y del código de referencia mediante la similitud de Jaccard aplicada sobre los multiconjuntos de tipos de nodos. Los valores que puede coger van de 0 hasta 1, cuanto más alto es este valor significa que más se ha aproximado a la estructura del código original.

Complementariamente, el node ratio relaciona el número total de nodos del AST del código generado con el del código de referencia, permitiendo cuantificar si el modelo ha simplificado o complejizado la estructura del programa. Valores superiores a 1 indican que el código generado es estructuralmente más complejo que el original, comportamiento esperable en la ofuscación, mientras que valores inferiores reflejan una simplificación.

En el plano funcional, la métrica de functional match evalúa si la salida producida por el código desofuscado coincide con la del programa original mediante ejecución real, mientras que el error rate recoge la proporción de casos en los que el código generado no llega siquiera a ejecutarse correctamente.

La combinación de métricas estructurales con métricas de ejecución funcional ofrece la caracterización más completa de la calidad del código generado, ya que ninguna de ellas es suficiente por sí sola: un código puede ser estructuralmente similar al original pero semánticamente incorrecto, o ejecutarse sin errores, pero producir resultados diferentes. Existe literatura que demuestra el uso de la similitud de Jaccard sobre tipos de nodos AST como métrica robusta y correlacionada con otras medidas de referencia en la evaluación de código generado por LLMs.ⁱⁱⁱ

4.3. Modelos de lenguaje a gran escala (LLMs)

4.3.1. Arquitectura general

Los LLMs son modelos de redes neuronales profundas que se entrenan sobre enormes volúmenes de información con el objetivo de predecir el siguiente en una secuencia de tokens. Su arquitectura se basa en el mecanismo “Transformer”, introducido por Vaswani en 2017, que sustituyó los modelos recurrentes secuenciales por un mecanismo capaz de procesar los tokens de forma paralela, lo que significa una mejora notable en la eficiencia de los procesos de aprendizaje.^{iv}

En la actualidad, estos modelos cuentan con versiones de miles de millones de parámetros y han sido entrenados sobre código fuente en múltiples lenguajes de programación de todo tipo, lo que les confiere una comprensión implícita de las convenciones sintácticas y semánticas del código.

El comportamiento de un LLM durante la generación no depende únicamente de su arquitectura y entrenamiento, sino también de una serie de parámetros de configuración que controlan el proceso de muestreo de tokens. Estos mecanismos son parte fundamental del funcionamiento de los modelos modernos.^v

-
- La temperatura regula la aleatoriedad de la distribución de probabilidad sobre el vocabulario: valores bajos producen respuestas más deterministas y conservadoras, mientras que valores altos favorecen la diversidad y la creatividad de la salida.
 - Por otro lado, las técnicas de *sampling* controlan qué tokens pueden ser seleccionados en cada paso de generación.

En particular, *top_k* restringe el muestreo a los k tokens más probables, mientras que *top_p* limita la selección al subconjunto mínimo de tokens cuya probabilidad acumulada supera un umbral p , descartando las opciones menos probables. Ambos mecanismos reducen la probabilidad de generar tokens poco plausibles, aunque *top_p* adapta dinámicamente el tamaño del conjunto candidato en función de la distribución de probabilidades.

- Por último, el *repeat penalty* penaliza la repetición de tokens ya generados, reduciendo la tendencia del modelo a producir secuencias redundantes.

La configuración conjunta de estos parámetros permite ajustar el equilibrio entre precisión y variabilidad de la salida, lo que resulta especialmente relevante en tareas como la ofuscación, donde se busca diversidad estructural, y la desofuscación, donde se prioriza la exactitud semántica.

4.3.2. Aplicaciones en ingeniería del software

Más allá de las tareas clásicas de procesamiento de lenguaje natural, los LLMs han demostrado una notable versatilidad en el ámbito de la ingeniería del software. Su aplicación más extendida es la generación de código a partir de descripciones en lenguaje natural, función que herramientas como Copilot han popularizado de forma masiva entre los desarrolladores. Sin embargo, abarca más campos se han empleado con éxito en tareas de depuración automática, documentación, refactorización, detección de vulnerabilidades, comprensión de código legado, entre muchas otras.

En el ámbito de la seguridad del software, existe una revisión sistemática que recoge el uso de LLMs en tareas tan diversas como la detección de malware, el análisis de binarios, la identificación de vulnerabilidades o la generación de exploits en pruebas de penetración controladas.^{vi}

Este conjunto de capacidades convierte a los LLMs en candidatos naturales para abordar el problema de la desofuscación de código, dado que dicha tarea requiere precisamente la combinación de comprensión semántica del código, razonamiento sobre flujos de control y la capacidad de recuperar intenciones programáticas a partir de representaciones sintácticamente degradadas.

4.4. Prompt engineering

4.4.1. Definición y relevancia

El prompt engineering es el conjunto de técnicas orientadas a formular las instrucciones que se suministran a un LLM de manera que maximicen la calidad y la adecuación de la respuesta generada. Lejos de ser un aspecto secundario, la formulación del prompt ha demostrado ser un factor determinante en el rendimiento de los modelos: variaciones aparentemente menores en la redacción de las instrucciones pueden producir diferencias sustanciales en la calidad de los resultados. En el contexto de este trabajo, el diseño de un buen prompt constituye un componente crítico del pipeline experimental, ya que la capacidad del modelo para desofuscar correctamente depende en gran medida de cómo se le presenta la tarea.

4.4.2. Estrategias principales

En la literatura se distinguen dos estrategias fundamentales de prompting que resultan especialmente relevantes para este trabajo: ^{vii}

- Zero-shot prompting: el modelo recibe únicamente la tarea que debe realizar, sin ningún ejemplo previo. Esta estrategia permite evaluar la capacidad intrínseca del modelo para generalizar a tareas no vistas, apoyándose exclusivamente en el conocimiento adquirido durante el preentrenamiento.
- Few-shot prompting: se proporcionan al modelo uno o varios ejemplos de entrada y salida esperados antes de presentarle la tarea real. Esta técnica, introducida formalmente por Brown et al. en el contexto de GPT-3, permite guiar el comportamiento del modelo y mejorar la consistencia del formato de salida, resultando especialmente útil cuando la tarea presenta una estructura bien definida.

Junto a estas dos estrategias, la literatura recoge también el chain-of-thought prompting, que instruye al modelo para que razone paso a paso antes de producir su respuesta final.

Aunque esta técnica ha demostrado mejoras notables en tareas de razonamiento complejo, su aplicación en el contexto de la desofuscación de código requiere una evaluación cuidadosa, ya que puede incrementar considerablemente el número de tokens generados sin garantizar una mejora proporcional en la calidad semántica del resultado. ^{viii}

5. Estado del arte

5.1. Capacidades de los LLMs en tareas de transformación de código

El interés por aplicar LLMs a tareas de transformación y análisis de código ha crecido notablemente en los últimos años. Sin embargo, la mayor parte de los trabajos existentes aborda estos problemas de forma fragmentada:

- Unos estudian si comprenden el código a nivel semántico
- Otros evalúan su capacidad para ofuscarlo
- Y otros su utilidad para revertir ese proceso

En conjunto, la literatura evidencia tanto un potencial prometedor como limitaciones estructurales que varían según la técnica, el modelo y el lenguaje de programación. Los siguientes subapartados revisan cada uno de estos ejes de forma progresiva.

5.1.1. Compresión semántica del código por parte de los LLMs

Un eje transversal a la ofuscación y la desofuscación es la capacidad de los LLMs para comprender el código a nivel semántico, no únicamente sintáctico.

Laneve aborda directamente esta cuestión en su trabajo sobre evaluación de la comprensión del código en LLMs, mostrando que los modelos fallan en reconocer equivalencias semánticas en aproximadamente el 41 % de los casos cuando no se proporciona contexto adicional. Este resultado es especialmente relevante para el presente trabajo, ya que tanto la ofuscación como la desofuscación requieren que el modelo preserve el comportamiento semántico del código transformado.^{ix}

En la misma dirección, Nikiema (2025) publica *The Code Barrier: What LLMs Actually Understand?*, un estudio que analiza en profundidad qué aspectos del código comprenden realmente los LLMs y cuáles se limitan a reproducir patrones superficiales. Sus conclusiones apuntan a que los modelos presentan dificultades sistemáticas con abstracciones de alto nivel y con transformaciones que alteran la forma, pero no la función, lo cual conecta directamente con los retos que plantea la ofuscación semántica.

x

5.1.2. LLMs aplicados a la desofuscación de código

La desofuscación mediante LLMs ha recibido atención creciente debido a la capacidad de estos modelos para razonar sobre la semántica del código más allá de su forma sintáctica.

El trabajo de referencia más relevante para es el de Hajipour et al. (2025), publicado en la conferencia DIMVA, que explora sistemáticamente la capacidad de los LLMs para

simplificar código ofuscado mediante transformaciones fuente a fuente. Sus experimentos con cinco técnicas de ofuscación distintas muestran que, a pesar de los desafíos en la corrección semántica, los LLMs pueden ser muy eficaces en esta tarea cuando se realiza un ajuste fino (fine-tuning) sobre cadenas de hasta siete transformaciones encadenadas, superando consistentemente a los compiladores optimizadores como línea de base. Este estudio utiliza un marco experimental con modelos de propósito general, metodología que comparte con el presente trabajo, aunque aquí se amplía el análisis a modelos locales de menor tamaño y se incluye también la tarea de ofuscación.^{xi}

En la misma línea, Patsakis et al. (2024) evalúan cuatro LLMs sobre scripts maliciosos reales de la campaña de malware Emotet, concluyendo que, aunque los modelos no alcanzan una precisión perfecta, algunos consiguen desofuscar cargas útiles de forma eficiente, lo que abre la puerta a su uso en pipelines de inteligencia de amenazas. Este estudio fue publicado posteriormente como artículo de revista en *Expert Systems with Applications* (2024).^{xii}

El trabajo más específico sobre JavaScript es JsDeObsBench (Chen et al., 2025), presentado en la conferencia ACM CCS 2025. Es el primer benchmark sistemático para medir la desofuscación de JavaScript por parte de LLMs, evaluando seis modelos de última generación (GPT-4o, Mixtral, Llama, DeepSeek-Coder, entre otros) frente a siete técnicas de ofuscación distintas. Sus resultados presentan tasas de fallo de ejecución medias del 37,40 %, mientras que los métodos tradicionales apenas fallan en este aspecto. Tanto los LLMs como las líneas de base son sensibles al número de transformaciones de ofuscación aplicadas en cadena. Este benchmark constituye el trabajo más cercano al presente TFG en cuanto al lenguaje de estudio y las métricas empleadas, y será utilizado como referente comparativo.^{xiii}

5.1.3. LLMs como generadores de código ofuscado

La dimensión generativa de la ofuscación mediante LLMs ha sido menos explorada que la desofuscación, pero ha recibido atención reciente por sus implicaciones de seguridad. Mohseni et al. (2025), en el marco de la conferencia AAAI 2025, desarrollan el benchmark MetamorphASM para evaluar si los LLMs pueden generar código ensamblador ofuscado. Con un corpus de 328.200 muestras y tres técnicas (inserción de código muerto, sustitución de registros y cambio de flujo de control), demuestran que modelos como GPT-4, LLaMA 3.1 y CodeLlama son capaces de producir código ofuscado funcionalmente equivalente al original, lo que representa un riesgo potencial para los motores antivirus. Sus métricas basadas en teoría de la información son complementarias a las métricas de similitud estructural y funcionalidad que se emplean en este trabajo.^{xiv}

Desde una perspectiva de seguridad ofensiva, LLMalMorph (2025) estudia la factibilidad de usar LLMs para generar variantes de malware a partir de código fuente, consiguiendo reducciones de detección de entre el 10 y el 15 % en motores antivirus y tasas de éxito de hasta el 91 % frente a detectores basados en aprendizaje automático. Este trabajo, aunque centrado en el dominio malicioso que queda fuera del alcance de este TFG, ilustra el potencial real de los LLMs para producir transformaciones que preservan el comportamiento funcional del código.^{xv}

5.2. Síntesis y posicionamiento del trabajo

La revisión del estado del arte permite identificar las siguientes tendencias:

1. La mayoría de los estudios existentes se centran en la desofuscación, siendo escasos los trabajos sobre la generación de código ofuscado mediante LLMs en lenguajes de alto nivel.
2. Los benchmarks disponibles utilizan principalmente modelos propietarios de gran tamaño o modelos de código especializados, mientras que apenas existen evaluaciones de modelos abiertos y locales de tamaño reducido.
3. El lenguaje JavaScript ha comenzado a recibir atención específica, pero no existen trabajos que evalúen simultáneamente ambas tareas en este lenguaje con los modelos seleccionados en este TFG.
4. La comparación sistemática con métodos tradicionales como línea de base es una práctica minoritaria en la literatura, y cuando se realiza, se hace en entornos de recursos computacionales elevados.

El presente trabajo se posiciona en este hueco: evalúa tanto la ofuscación como la desofuscación de JavaScript mediante cuatro LLMs locales de acceso abierto y tamaño reducido, con dos estrategias de prompting distintas, y compara sistemáticamente los resultados con la ofuscación tradicional como referencia. Esto permite aportar evidencia empírica reproducible sobre las capacidades y limitaciones de los LLMs en un escenario de hardware accesible, contribuyendo a rellenar un vacío en la literatura identificado en la revisión anterior.

6. Desarrollo del framework

6.1. Arquitectura del sistema

El programa que se ha desarrollado consiste en una aplicación de consola en la cual el usuario trabaja con una serie de paneles y pantallas guiados de forma sencilla e interactiva.

Para poder ejecutarlo, es necesario clonar el repositorio correspondiente de GitHub en el equipo. Adjunto a continuación el enlace:

<https://github.com/GabriLog/llm-code-obfuscation-framework>

La arquitectura del sistema se divide en dos bloques principales:

- **Módulo de ejecución:** responsable de la selección del experimento, ejecución de estos y la generación de logs.
- **Módulo de evaluación:** encargado de procesar, calcular métricas y poder evaluar estos logs generados.

El punto de entrada del framework es el archivo script llamado “app”, ubicado en el directorio principal. Este acepta los dos modos de ejecución mediante parámetros de línea de comandos.

En su modo principal, es decir, sin emplear flags adicionales, el programa abre un flujo de selección del experimento que una vez el usuario ha rellenado comienza la ejecución.

Con el flag -ev, se activa el módulo de evaluación sobre los logs, este no requiere intervención del usuario, se debe ejecutar cuando ya se han realizado todos los experimentos de interés.

La ejecución del framework se lanza directamente desde la raíz del repositorio mediante el comando python app.py o py app.py en entornos Windows.

6.2. Estructura del repositorio

El proyecto se organiza en la siguiente estructura de directorios:

Directorio / Archivo	Contenido y propósito
core/	Presenta la funcionalidad principal del programa, dividiéndose en la interfaz de usuario, dominio, utilidades, entre otros.
datasets/	Contiene los scripts originales y sus versiones ofuscadas, organizados por tamaño.
prompts/	Plantillas de prompts para las tareas de ofuscar y desofuscar
outputs/	Contiene los resultados y gráficos obtenidos tras haber ejecutado ya el módulo de evaluación
logs/	Registros de las ejecuciones de modelos con información de tiempo, experimento y desempeño de cada uno
runs/	Encargado de configurar los parámetros de ejecución para LLMs y métodos tradicionales
app.py	Punto de entrada del sistema que orquesta el resto de módulos
prompts_config.json	Establece la asignación de prompts a modelos y tareas específicas
requirements.txt	Dependencias Python del proyecto

Tabla 2. Estructura del repositorio

6.3. Flujo de ejecución

Antes de comenzar con el flujo del programa, es imprescindible haber instalado los modelos con los que deseamos trabajar en Ollama con anterioridad a la ejecución del programa para que estos salgan disponibles y poder ejecutarlos.

Teniendo eso en cuenta, el flujo consistiría en los siguientes puntos:

6.3.1. Selección del experimento

Todo experimento está compuesto por los siguientes datos que el usuario deberá introducir uno por uno:

- Modelo (mistral, qwen...)
- Estrategia (zero-shot/few-shot)
- Dataset (short/large)
- Script (por ej. factorial.js)

En función de lo seleccionado se recibirá como entrada el prompt correspondiente en el fichero de configuración de prompts. En caso de querer variar este fichero, se deben realizar los cambios e iniciar un nuevo experimento con estos ya aplicándose. Lo mismo pasa con los parámetros de configuración de los LLMs.

Esto tardará una serie de segundos o minutos en función de lo que tarde el modelo específico en procesar y dar respuesta al prompt que se le está pasando.

6.3.2. Generación de logs

Por cada experimento que ejecutemos se generará un log con la fecha y hora en que ha sido generado, con la finalidad de poder llevar un seguimiento del histórico de experimentos ejecutados en el framework.

Cada uno de los logs estará compuesto por la siguiente información:

- Timestamp
- Resumen del experimento
- Contenido del script original
- Resultado de ofuscación tradicional
- Prompt de ofuscación para el modelo
- Resultado de ofuscación del modelo
- Prompt de desofuscación para el modelo
- Resultado de desofuscación del modelo

6.3.2. Evaluación de resultados

Una vez los logs ya han sido creados pasaríamos a la evaluación, se obtendrán una serie de resultados a partir de todos los logs de interés generados:

- AST score
- Node ratio
- Error rate
- Acierto funcional
- Tiempo de ejecución

6.3.4. Representación de gráficas

A partir de ello, se procede a una agregación de métricas para su visualización.

- Tasas de error y acierto funcional
- Impacto de zero-shot y few-shot
- Relación entre complejidad y funcionalidad
- Tiempos medios de ejecución

7. Experimentación

7.1. Diseño experimental

El diseño experimental se basa en una evaluación sistemática de los modelos seleccionados bajo distintas configuraciones controladas, combinando variables clave de la generación y los datos de entrada.

Se consideran los siguientes factores:

- Cuatro modelos: llama3.2:3b, llama3.1:8b, qwen2.5-coder:7b y mistral:7b
- Dos estrategias de prompting: zero-shot y few-shot
- Dos tareas: ofuscación y desofuscación
- Seis scripts del dataset: tres de tipo short y tres de tipo large

La combinación de estos factores da lugar a un total de:

$$4 \times 2 \times 2 \times 6 = 96 \text{ experimentos distintos posibles}$$

En el presente trabajo se ha realizado al menos una vez cada uno de ellos.

Cada ejecución constituye un experimento independiente, generando su información detallada sobre el comportamiento del modelo, incluyendo métricas estructurales, funcionales y de rendimiento.

De forma adicional, se ejecuta la ofuscación tradicional sobre el mismo conjunto de scripts, proporcionando una línea base de referencia con la que comparar los resultados obtenidos por los modelos.

Este diseño permite analizar de forma controlada el impacto de cada una de las variables consideradas, facilitando la comparación directa entre modelos, estrategias y tamaños de código, así como la identificación de patrones de comportamiento en los distintos escenarios evaluados.

7.2. Modelos evaluados

La selección combina intencionalmente modelos de propósito general y modelos especializados, con el fin de analizar el objetivo específico de si la especialización en código proporciona un añadido de valor sobre los procesos de ofuscación y desofuscación.

Los cuatro modelos evaluados representan un espectro de capacidades, tipos y tamaños:

Modelo	Parámetros	Especialización	Justificación
llama3.2:3b	3B	General	Modelo pequeño; referencia de bajo coste computacional
llama3.1:8b	8B	General	Versión mayor de LLaMA 3; mayor capacidad de razonamiento
qwen2.5-coder:7b	7B	Código	Modelo especializado en tareas de programación
mistral:7b	7B	General	Modelo general de alto rendimiento en tareas de código

Tabla 3. LLMs evaluados

La menor capacidad en tamaño de llama3.2:3b nos servirá de guía para observar las limitaciones de rendimiento en contextos de menor de coste computacional. Llama3.1:8b proporciona mayor capacidad de razonamiento y comprensión del código respecto a su versión reducida, comparativa directa. Qwen2.5-coder:7b es el representante de los modelos especializados en programación en la evaluación. Mistral:7b ha demostrado un rendimiento competitivo en benchmarks de código a pesar de no estar específicamente ajustado para este dominio.

7.3. Definición de datasets

El dataset utilizado en los experimentos está compuesto por un conjunto de scripts escritos en JavaScript, diseñados manualmente y organizados en función de su tamaño y complejidad.

Se distinguen dos categorías:

- Scripts short: compuestos por entre 10 y 20 líneas de código
- Scripts large: compuestos por entre 40 y 60 líneas de código

Cada categoría contiene tres scripts, dando lugar a un total de seis programas base. Para cada uno de ellos se ha generado su correspondiente versión ofuscada mediante técnicas tradicionales, que se utiliza tanto como entrada en los experimentos de desofuscación como referencia para la evaluación de la ofuscación generada por los modelos.

Los scripts cubren distintas tipologías de problemas algorítmicos habituales, incluyendo operaciones matemáticas, generación y análisis de estructuras de datos, y lógica de control mediante condicionales y bucles. Entre los ejemplos incluidos se encuentran el cálculo del factorial, la secuencia de Fibonacci, la comprobación de números primos, el análisis de conjuntos numéricos, la generación y análisis de matrices y una calculadora básica con múltiples operaciones.

Esta selección permite evaluar el comportamiento de los modelos sobre patrones representativos del código JavaScript, manteniendo al mismo tiempo un entorno controlado en el que es posible verificar de forma precisa la correctitud funcional de los resultados.

El tamaño del dataset se ha definido teniendo en cuenta el coste computacional asociado a la ejecución de los experimentos, ya que cada interacción con los modelos puede requerir varios minutos de procesamiento. A pesar de ello, el número total de ejecuciones realizadas garantiza una cobertura completa del espacio experimental definido, permitiendo obtener resultados consistentes y comparables entre las distintas configuraciones evaluadas.

Asimismo, al tratarse de scripts diseñados específicamente para el experimento, se dispone de un conocimiento completo sobre su comportamiento esperado, lo que facilita la evaluación mediante ejecución real y la detección precisa de errores funcionales en las transformaciones generadas.

7.4. Configuración experimental

7.3.1. Diseño de prompts

Los prompts creados se han escrito en inglés, al tratarse del idioma que mejor interpretan estos modelos y, por tanto, mejor rendimiento obtienen.

Para la tarea de ofuscación, el prompt instruye al modelo para que transforme el código proporcionado aplicando técnicas avanzadas de ofuscación, tales como el renombrado sistemático de identificadores, el uso de literales en formato hexadecimal, la codificación de cadenas, la introducción de funciones auxiliares e indirectas, así como la utilización de estructuras como tablas de búsqueda o transformaciones del flujo de control. Asimismo, se imponen restricciones estrictas de salida, limitando la respuesta exclusivamente a código JavaScript sin explicaciones, comentarios ni formato adicional.

{generic_obfuscation.txt}

You are an expert in advanced JavaScript obfuscation.

Heavily obfuscate the following code while preserving EXACT runtime behavior.

Requirements:

- Rename all identifiers to meaningless names*
- Use hexadecimal numeric literals*
- Convert string literals into encoded representations (e.g., base64 or char codes)*
- Introduce indirection through helper functions*
- Use array-based string lookup tables*
- Apply control flow flattening when possible*
- Avoid clear loop structures if possible*

The result should be extremely difficult for a human to understand.

CRITICAL:

- *Output ONLY raw JavaScript code.*
- *Do NOT include explanations.*
- *Do NOT include comments.*
- *Do NOT use markdown.*
- *Do NOT include backticks.*

Code:

{code}

Para la tarea de desofuscación, el prompt solicita al modelo que recupere una versión clara, legible y estructurada del código ofuscado, preservando exactamente su comportamiento original. Para ello, se le indica que restaure nombres de identificadores con significado, reemplace literales codificados por representaciones comprensibles, simplifique el flujo de control eliminando niveles innecesarios de indirección y suprima código muerto que no afecte a la ejecución.

{generic_deobfuscation.txt}

You are an expert JavaScript reverse engineer specializing in deobfuscation.

Completely deobfuscate the following JavaScript code.

Requirements:

- *Preserve EXACT runtime behavior.*
- *Do NOT change logic, side effects, or edge case handling.*
- *Restore meaningful identifier names.*
- *Replace hexadecimal and encoded literals with readable equivalents.*
- *Simplify control flow (remove unnecessary indirection, flattening, and lookup tables).*
- *Remove dead code if it does not affect runtime behavior.*
- *Produce clean, idiomatic, well-structured JavaScript.*

CRITICAL:

- *Output ONLY raw JavaScript code.*
- *Do NOT include explanations.*
- *Do NOT include comments.*
- *Do NOT use markdown.*
- *Do NOT include backticks.*

Code:

{code}

En el caso de la estrategia few-shot, incluye además ejemplos representativos de código original y su versión ofuscada correspondiente, obtenidos del propio dataset, estos se añaden a través del módulo `prompt_builder`.

EXAMPLE INPUT:

{example_original}

EXAMPLE OUTPUT:

{example_obfuscated}

La configuración de qué prompt se asigna a cada modelo y tarea se gestiona mediante el archivo `prompts_config.json`, lo que permite ajustar la asignación sin modificar el código fuente del framework.

7.3.2. Parámetros LLM

Dado que las tareas de ofuscación y desofuscación presentan requisitos distintos, se han definido configuraciones diferenciadas para cada una de ellas.

En la tarea de ofuscación se busca generar transformaciones variadas del código original, fomentando la diversidad estructural y sintáctica. Para ello se emplea una configuración que introduce aleatoriedad controlada en la generación, permitiendo al modelo explorar diferentes formas de transformación sin comprometer la coherencia del código producido:

- `temperature = 0.35`: valor moderado que introduce cierta variabilidad en la generación. No es lo suficientemente bajo como para producir siempre la misma transformación, ni lo suficientemente alto como para generar código incoherente. Permite que el modelo explore distintas estrategias de ofuscación manteniendo la estructura lógica del programa.
- `top_p = 0.95`: umbral permisivo que amplía el conjunto de tokens candidatos al 95 % de la masa de probabilidad acumulada. Favorece la diversidad léxica y estructural en el código generado, lo que resulta deseable en la ofuscación, donde se busca que el resultado difiera del original.
- `top_k = 60`: se mantienen los 60 tokens más probables en cada paso de generación. Es un valor moderado-alto que amplía el espacio de opciones disponibles sin introducir tokens marginales que pudieran comprometer la validez sintáctica del código.
- `repeat_penalty = 1.05`: penalización leve sobre los tokens ya generados. Suficiente para evitar repeticiones triviales, pero lo bastante baja para no impedir que el modelo repita estructuras que son propias del código ofuscado, como patrones de renombrado o cadenas codificadas.
- `num_predict = 3000`: límite máximo de tokens en la respuesta. Dimensionado para que el código ofuscado —que tiende a ser más extenso que el original debido a la inserción de código auxiliar y la expansión de expresiones— quepa íntegro sin que la respuesta sea truncada.

En la tarea de desofuscación el objetivo es recuperar una versión clara y correcta del código, preservando exactamente su comportamiento original. Se prioriza un

comportamiento determinista y preciso, reduciendo al mínimo la variabilidad en la salida. Los parámetros se ajustan en consecuencia:

- `temperature = 0.02`: valor casi cero que hace que el modelo sea prácticamente determinista, seleccionando en cada paso el token más probable. Esto minimiza la creatividad y favorece respuestas estables y reproducibles, lo cual es crítico cuando se exige exactitud semántica en el código recuperado.
- `top_p = 0.85`: umbral más restrictivo que en la ofuscación: el modelo solo considera los tokens que acumulan el 85 % de la probabilidad, descartando opciones menos probables. Reduce la variabilidad en la elección de tokens y orienta la generación hacia las construcciones de código más convencionales y predecibles.
- `top_k = 40`: conjunto de candidatos más estrecho que en la ofuscación. Al limitar la selección a los 40 tokens más probables, el modelo se ciñe a las opciones más seguras en cada paso, reduciendo el riesgo de introducir variaciones no deseadas en el código recuperado.
- `repeat_penalty = 1.2`: penalización más elevada que en la ofuscación. Desincentiva con mayor fuerza la generación de bloques de código redundantes, un problema frecuente en la desofuscación cuando el modelo tiende a reproducir fragmentos ya generados o a entrar en bucles de salida.
- `num_predict = 4000`: margen de tokens superior al de la ofuscación. El código desofuscado tiende a ser más extenso y estructurado que el ofuscado —al reintroducir nombres descriptivos, comentarios y estructuras de control explícitas—, por lo que se amplía el límite para evitar truncamientos en scripts de mayor tamaño.

La diferencia entre ambas configuraciones refleja directamente la naturaleza opuesta de las dos tareas: mientras que la ofuscación requiere diversidad y creatividad controlada, la desofuscación demanda precisión y consistencia. Esta dualidad en la configuración es uno de los aspectos metodológicos que distingue el diseño experimental de este trabajo.

8. Métricas y evaluación

8.1. Métricas

Para poder evaluar de forma rigurosa el rendimiento de los modelos en las tareas de ofuscación y desofuscación, el framework desarrollado implementa un conjunto de métricas complementarias que permiten caracterizar tanto la calidad estructural del código generado como su comportamiento funcional. Cada métrica aporta una dimensión diferente del análisis y, en conjunto, ofrecen una visión completa del desempeño de cada modelo.

El módulo de evaluación procesa de forma automatizada los logs generados tras cada experimento y calcula estas métricas de manera sistemática para todos los modelos, estrategias y scripts del dataset. Los resultados individuales se agregan posteriormente en gráficas comparativas que permiten identificar patrones de comportamiento a lo largo de las distintas configuraciones evaluadas.

A continuación, se describen en detalle las métricas empleadas y su justificación en el contexto de este trabajo.

8.1.1. Similitud estructural: AST score y node ratio

La similitud estructural mide en qué medida el código generado por el modelo mantiene o altera la estructura del programa original. Para ello se utilizan dos métricas complementarias derivadas del Abstract Syntax Tree del código, construido mediante la librería Esprima.

El AST score cuantifica la similitud entre el AST del código generado y el del código de referencia, mediante la comparación del conjunto de tipos de nodos presentes en ambos árboles. Su cálculo se basa en la similitud de Jaccard aplicada sobre los multiconjuntos de tipos de nodos:

$$AST\ score = |A \cap B| / |A \cup B|$$

Donde A y B representan los multiconjuntos de tipos de nodos del AST del código generado y del código de referencia, respectivamente. El resultado es un valor en el rango [0, 1], donde 1 indica identidad estructural completa y 0 indica ausencia total de nodos compartidos.

En el contexto de la ofuscación, el código de referencia es el código original, ya que el objetivo es que el código ofuscado conserve la misma lógica con una estructura alterada; se espera por tanto que el AST score sea moderado o bajo, reflejando la transformación realizada. En el caso de la desofuscación, el código de referencia es también el código original, dado que el objetivo es recuperar la estructura original a partir de su versión ofuscada; en este caso se espera que el AST score sea alto, indicando una buena recuperación estructural.

El *node ratio* complementa el AST score midiendo la relación entre el número total de nodos del AST del código generado y el del código de referencia:

$$\text{node ratio} = \text{nodos}(\text{generado}) / \text{nodos}(\text{referencia})$$

Un valor cercano a 1 indica que el código generado tiene una complejidad estructural similar a la de referencia. Valores muy superiores a 1 indican que el código generado es considerablemente más complejo, lo que es un comportamiento esperado en la ofuscación. En la desofuscación, valores próximos a 1 indican una correcta simplificación del código.

Estas métricas son especialmente adecuadas para el lenguaje JavaScript, cuya naturaleza dinámica hace que el análisis léxico superficial no capture de forma fiable la complejidad real del código. El uso del AST permite una comparación semánticamente más fundamentada que la simple comparación de líneas o caracteres.

8.1.1. Funcionalidad: error rate y functional match

La similitud estructural no es por sí sola un indicador suficiente de la calidad del código generado: un programa puede presentar un AST score elevado y aun así ser sintácticamente incorrecto o producir resultados diferentes a los esperados. Por esta razón, el framework incorpora métricas de funcionalidad que evalúan el comportamiento real del código mediante ejecución.

Para cada experimento, el módulo de evaluación ejecuta el código generado mediante Node.js en un entorno controlado, comparando su salida con la del script original. A partir de este proceso se obtienen dos métricas:

- **Error rate:** proporción de experimentos en los que la ejecución del código generado produce un error, ya sea de sintaxis, de tiempo de ejecución o de tipo, impidiendo la obtención de un resultado. Se calcula como el cociente entre el número de ejecuciones fallidas y el total de experimentos realizados para cada modelo y tarea.
- **Functional match:** proporción de experimentos en los que la salida producida por el código generado coincide exactamente con la del script original. Se calcula como el cociente entre el número de aciertos funcionales y el total de experimentos no fallidos para ese modelo y tarea.

El diseño de los scripts del dataset, que cubren operaciones matemáticas, generación de estructuras de datos y lógica de control, permite verificar de forma determinista la correctitud funcional: dado que los scripts no dependen de entrada del usuario ni de estados externos, la comparación de salidas es directa y no ambigua.

La combinación de error rate y functional match ofrece una caracterización completa del rendimiento funcional: un modelo puede tener un error rate bajo pero también un functional match bajo, lo que indicaría que genera código ejecutable pero semánticamente incorrecto.

8.1.3. Rendimiento: tiempo de ejecución

Además de la calidad del código generado, el framework registra el tiempo de ejecución de cada experimento, desde el envío del prompt al modelo hasta la recepción de la respuesta completa. Esta métrica se calcula de forma independiente para cada modelo y tarea, expresándose en segundos.

El tiempo de ejecución es una dimensión crítica para evaluar la viabilidad de los LLMs en escenarios de uso real. En el presente trabajo, todos los modelos se ejecutan de manera local mediante Ollama en hardware de uso personal, lo que hace que esta métrica refleje el coste computacional real en un entorno accesible y reproducible.

8.1.4. Línea de base: ofuscación tradicional

Para que los resultados de los modelos puedan ser interpretados en perspectiva, el framework ejecuta también la ofuscación tradicional sobre el mismo conjunto de scripts del dataset mediante JavaScript Obfuscator. Los resultados de esta ejecución se registran y procesan con las mismas métricas que los generados por los LLMs, proporcionando una línea de base de referencia con la que comparar directamente el rendimiento de cada modelo.

La ofuscación tradicional actúa como el techo de rendimiento esperado para la tarea de ofuscación, ya que se trata de un método determinista, diseñado específicamente para esta función y probado en producción. Su inclusión en la evaluación permite contextualizar las limitaciones y posibilidades de los LLMs frente a un método consolidado, alineándose con la recomendación metodológica planteada por el tutor de presentar los valores absolutos junto a los porcentajes para que estos no parezcan arbitrarios.

8.2. Evaluación de resultados

8.2.1. Similitud estructural y calidad del código generado

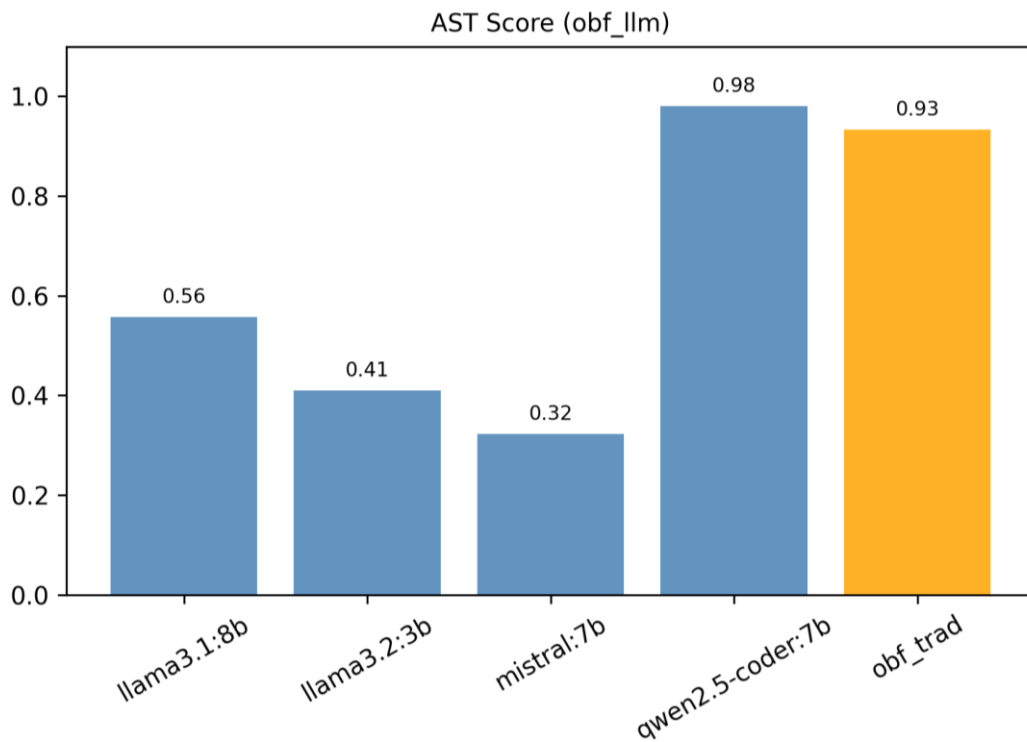


Ilustración 1. AST Score en la ofuscación

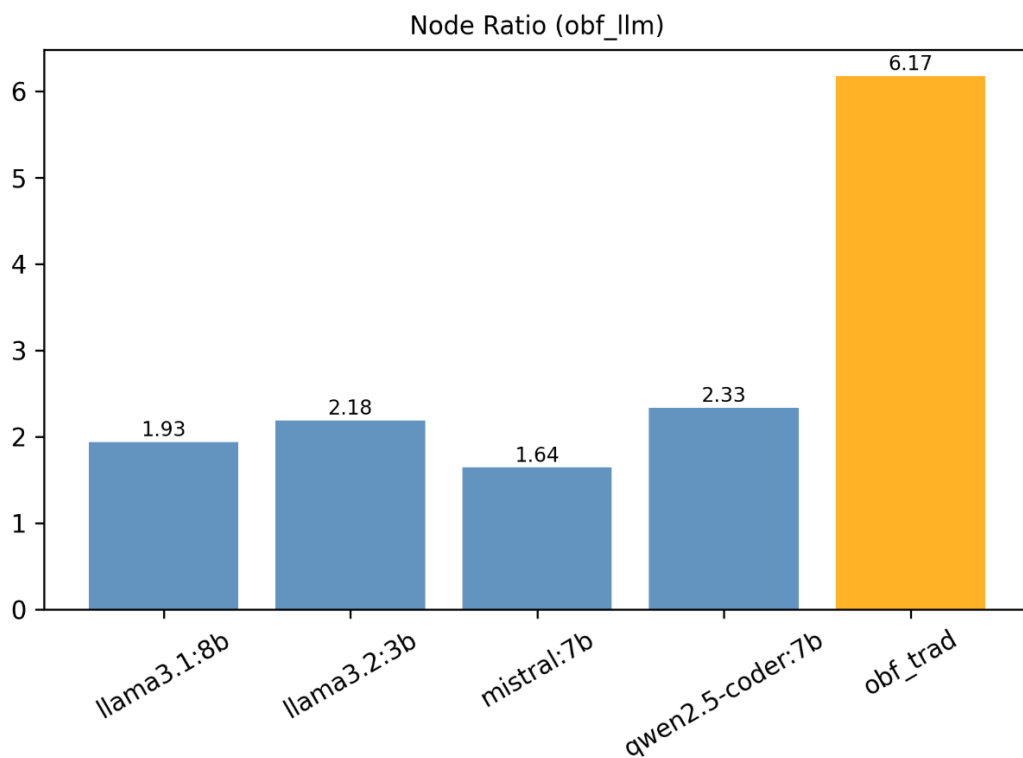


Ilustración 2. Node ratio en la ofuscación

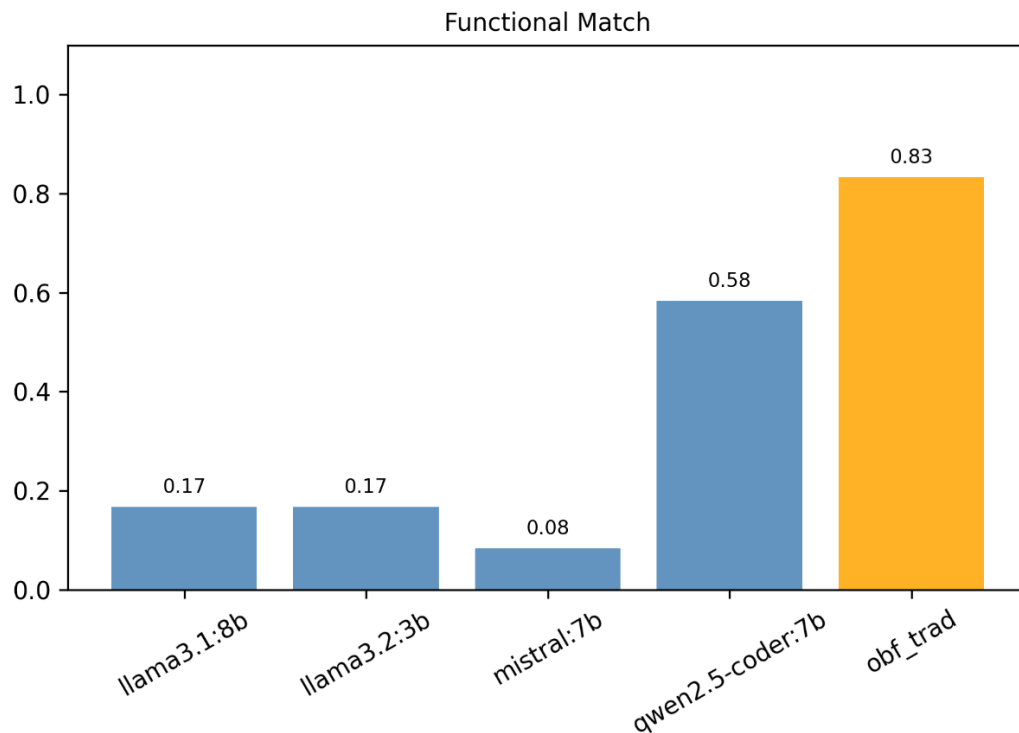


Ilustración 3. Functional match en la ofuscación

Modelo	AST score	Node ratio	Error rate	Functional match
Tradicional	0.93	6.17	0.0%	83.3%
qwen2.5-coder:7b	0.98	2.33	8.3%	58.3%
llama3.1:8b	0.56	1.93	41.7%	16.7%
llama3.2:3b	0.41	2.18	66.7%	16.7%
mistral:7b	0.32	1.64	83.3%	8.3%

Tabla 4. Tabla comparativa de ofuscación

La tabla recoge las métricas de similitud estructural y funcionalidad agregadas por modelo para la tarea de ofuscación, incluyendo la ofuscación tradicional como referencia. Para cada modelo se realizaron 12 experimentos de ofuscación (2 estrategias × 6 scripts), al igual que la ofuscación tradicional.

Los datos de la Tabla 4 muestran de forma clara el dominio de la ofuscación tradicional frente a los LLMs evaluados. En los 12 experimentos realizados para cada modelo, qwen2.5-coder:7b es el único LLM que se aproxima a un comportamiento aceptable, con un AST score de 0.98 y un error rate del 8.3% (1 fallo en 12 experimentos), aunque su funcional match (58.3%, es decir, 7 de 12 scripts) se queda 25 puntos por debajo del método tradicional. Su node ratio de 2.33 indica que sí produce una expansión estructural del código, aunque muy inferior a la del método clásico (6.17), lo que refleja que la transformación generada es superficial en comparación.

El resto de modelos presentan un deterioro progresivo y significativo. llama3.1:8b falla en 5 de 12 experimentos (41.7% de error rate) y solo logra un funcional match del 16.7% (2 de 12 scripts correctos), con un AST score de 0.56 que indica que la mayor parte del código generado presenta desviaciones estructurales notorias respecto al original. llama3.2:3b agrava este patrón: con un error rate del 66.7% (8 fallos en 12) y un AST score de 0.41, las transformaciones producidas son con frecuencia incorrectas o inutilizables. mistral:7b es el modelo con peor desempeño global, fallando en 10 de los 12 experimentos (83.3%) y alcanzando un funcional match de apenas el 8.3% (1 script correcto de 12), con el node ratio más bajo (1.64), lo que sugiere que sus salidas apenas modifican la estructura del código de entrada.

8.2.2. Resultados por tarea: ofuscación y desofuscación

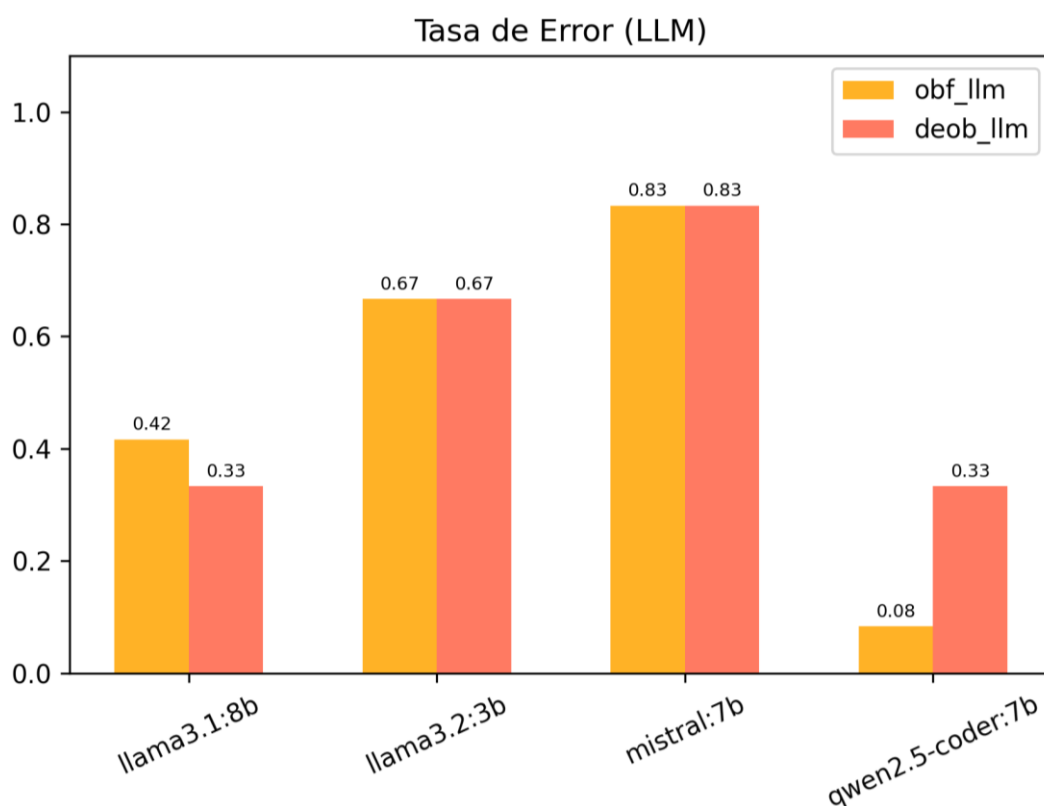


Ilustración 4. Tasa de error por estrategia

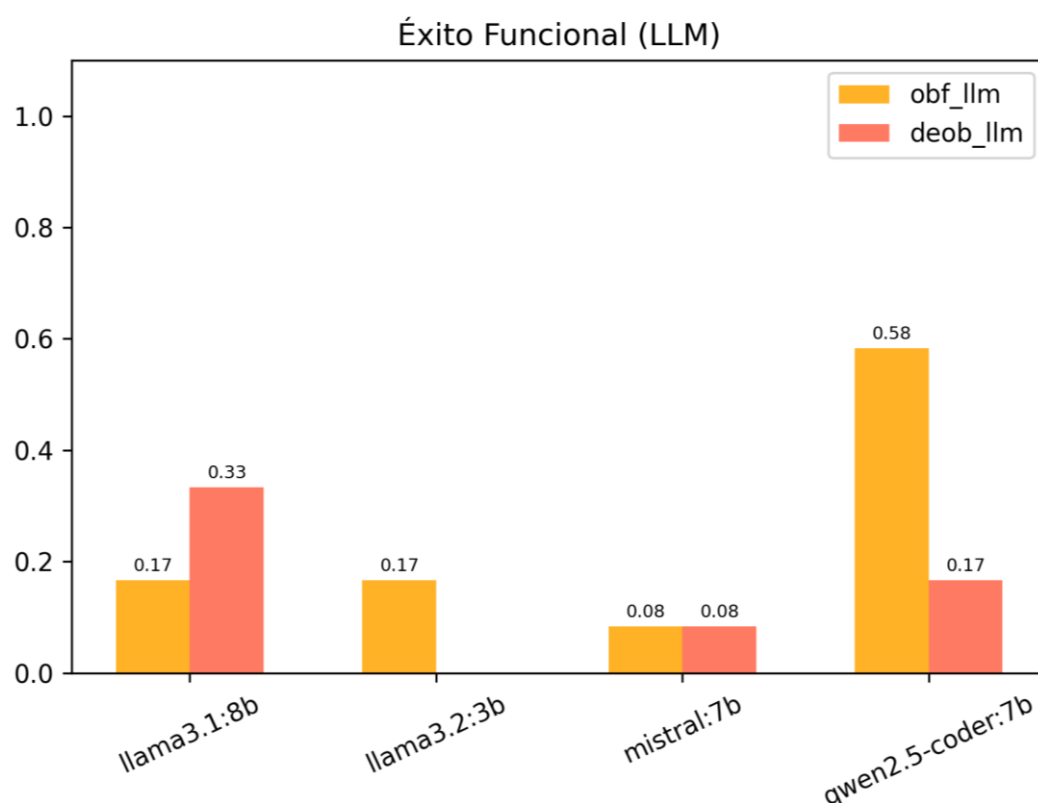


Ilustración 5. Éxito funcional por estrategia

Modelo	Ofuscación LLM		Desofuscación LLM	
	Error rate	Func. match	Error rate	Func. match
Tradicional	0.0%	83.3%	-	-
qwen2.5-coder:7b	8.3%	58.3%	33.3%	16.7%
llama3.1:8b	41.7%	16.7%	33.3%	33.3%
llama3.2:3b	66.7%	16.7%	66.7%	0.0%
mistral:7b	83.3%	8.3%	83.3%	8.3%

Tabla 5. Tabla comparativa de éxito funcional por tarea

La Tabla 5 desglosa los resultados por tarea para cada modelo. En total, cada modelo realizó 24 experimentos: 12 de ofuscación y 12 de desofuscación (2 estrategias × 6 scripts por tarea).

La ofuscación tradicional se incluye como referencia únicamente en la tarea de ofuscación, ya que no aplica a la desofuscación. Los datos confirman que la desofuscación es sistemáticamente más difícil para todos los modelos: incluso qwen2.5-coder:7b, que logra un funcional match del 58.3% en ofuscación (7 de 12 scripts correctos), cae al 16.7% en desofuscación (2 de 12). llama3.2:3b no produce ningún resultado funcionalmente correcto en desofuscación (0.0% sobre 12 experimentos).

La única excepción parcial es llama3.1:8b, que mejora ligeramente en desofuscación respecto a la ofuscación (33.3% frente a 16.7%), lo que sugiere que este modelo puede razonar mejor sobre código ya transformado que sobre la generación de transformaciones nuevas.

8.2.3. Impacto de la estrategia de prompting

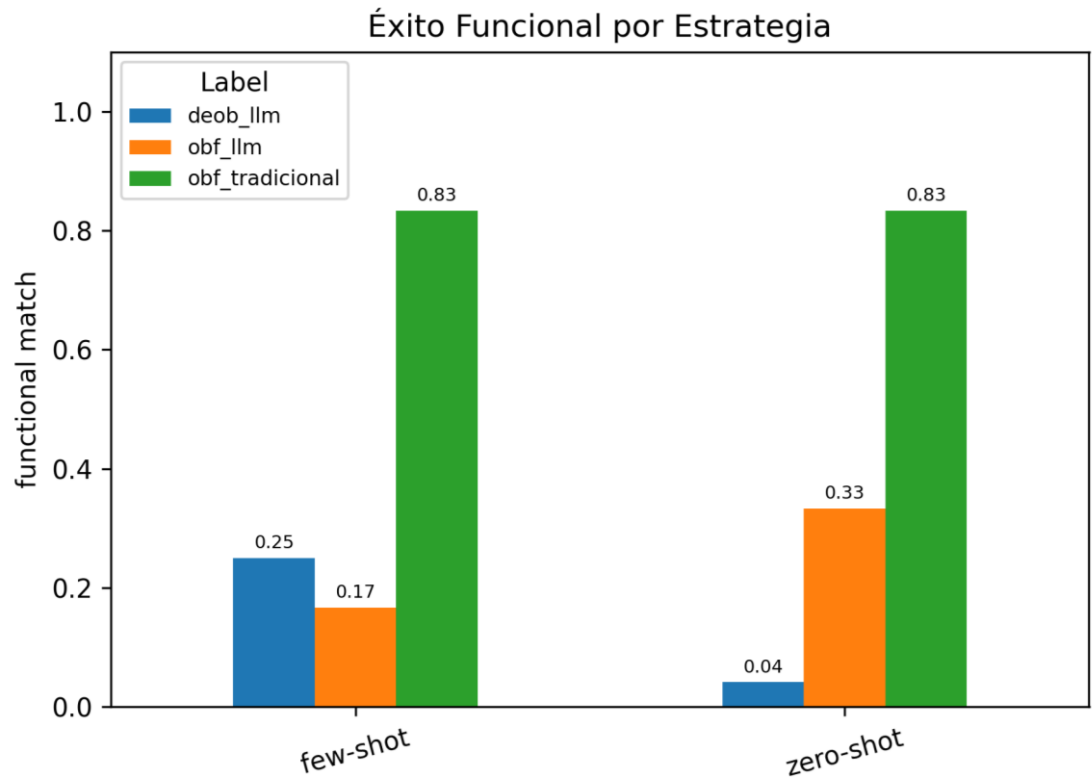


Ilustración 6. Éxito funcional por estrategia

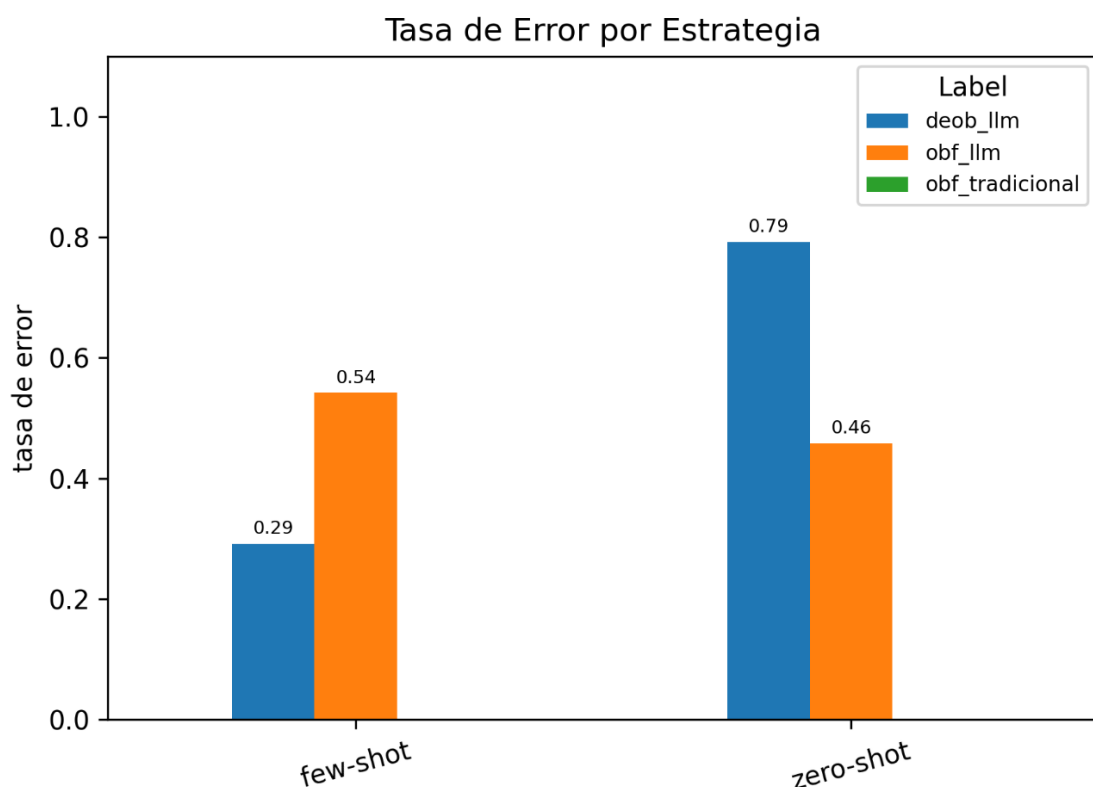


Ilustración 7. Tasa de error por estrategia

Modelo	Ofuscación LLM		Desofuscación LLM	
	Error rate	Func. match	Error rate	Func. match
Zero-shot	45.8%	33.3%	79.2%	4.2%
Few-shot	54.2%	16.7%	29.2%	25.0%

Tabla 6. Tabla comparativa de éxito funcional por estrategia

La Tabla 6 resume el impacto de la estrategia de prompting sobre los 48 experimentos realizados por estrategia (4 modelos $\times$ 2 tareas $\times$ 6 scripts). En la ofuscación, el zero-shot obtiene mejores resultados que el few-shot: 45.8% de error rate frente al 54.2% (22 fallos en 48 vs 26 fallos en 48), y un funcional match del 33.3% frente al 16.7% (16 éxitos en 48 vs 8 en 48).

Este resultado contraintuitivo sugiere que los ejemplos proporcionados en el few-shot actúan como un sesgo que lleva al modelo a imitar la forma del ejemplo en lugar de transformar genuinamente el script objetivo.

En la desofuscación, el patrón se invierte de forma notable: el few-shot reduce el error rate del 79.2% al 29.2% (de 38 fallos en 48 a 14 fallos en 48) y eleva el funcional match del 4.2% al 25.0% (de 2 aciertos en 48 a 12 en 48). En este caso, los ejemplos del prompt parecen proporcionar una referencia estructural que ayuda al modelo a comprender el tipo de transformación inversa esperada, especialmente en una tarea donde la ambigüedad de la instrucción zero-shot es mayor.

8.2.4. Tiempos de ejecución

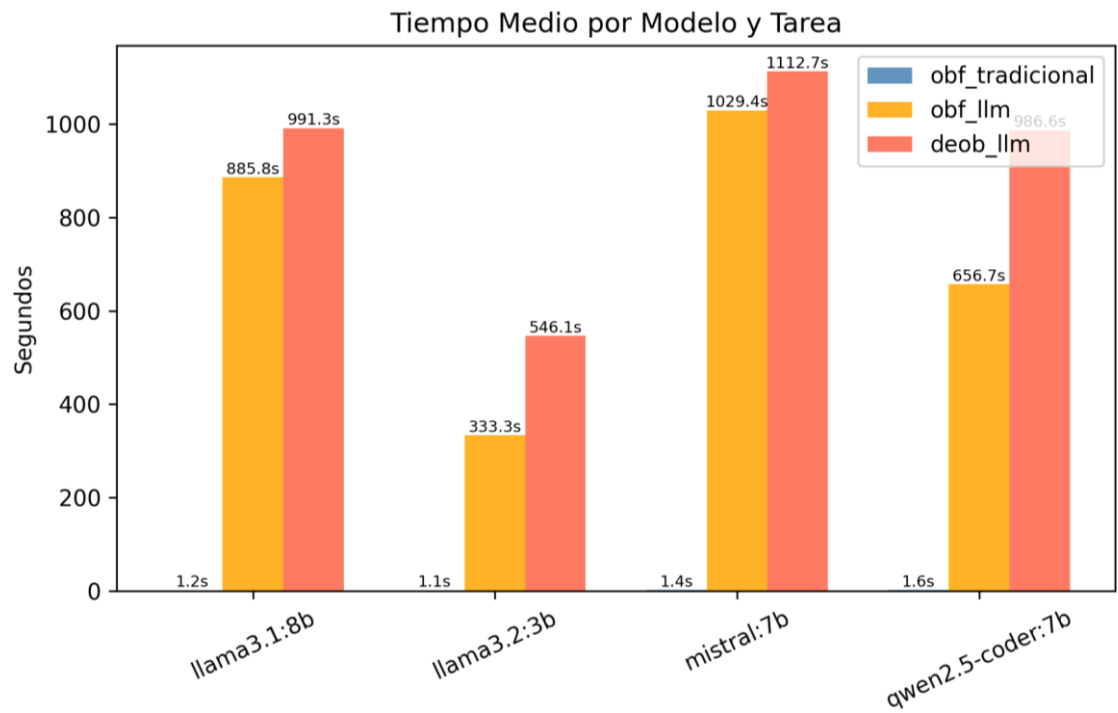


Ilustración 8. Tiempo medio por modelo y tarea

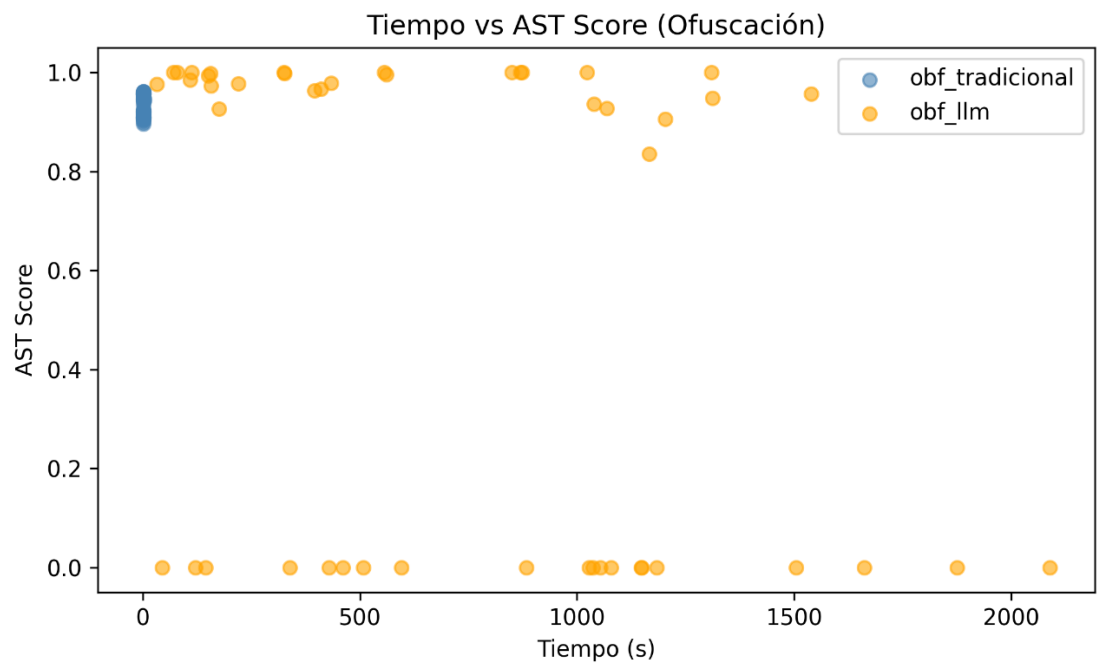


Ilustración 9. Mapa tiempo de ejecución/ast score

Modelo	Ofusc. tradicional (s)	Ofusc. LLM (s)	Desofusc. LLM (s)
llama3.1:8b	1.25	885.8	991.3
llama3.2:3b	1.14	333.3	546.1
mistral:7b	1.38	1029.4	1112.7
qwen2.5-coder:7b	1.57	656.7	986.6

Tabla 7. Tabla comparativa de tiempo medio de ejecución

La Tabla 7 recoge los tiempos medios de ejecución por modelo y tarea, calculados sobre los 12 experimentos de cada combinación. La brecha entre la ofuscación tradicional y los LLMs es de varios órdenes de magnitud: mientras que JavaScript Obfuscator completa cada script en torno a 1–2 segundos, los LLMs requieren entre 333.3 s y 1112.7 s por experimento, lo que equivale a entre 5 y 15 minutos. Mistral:7b es el modelo más lento en ambas tareas (1029.4 s en ofuscación y 1112.7 s en desofuscación), seguido de llama3.1:8b (885.8 s y 991.3 s respectivamente). qwen2.5-coder:7b, a pesar de ser el modelo con mejor desempeño cualitativo, presenta también un tiempo elevado (656.7 s en ofuscación y 986.6 s en desofuscación).

El modelo más rápido es llama3.2:3b (333.3 s en ofuscación y 546.1 s en desofuscación), lo cual resulta coherente con su menor número de parámetros (3b). En todos los casos, la desofuscación consume más tiempo que la ofuscación para el mismo modelo, lo que puede explicarse por la mayor complejidad semántica de la tarea inversa.

El mapa de la Ilustración 9 permite visualizar de forma conjunta este compromiso entre tiempo y calidad estructural, poniendo de manifiesto que no existe una correlación directa entre mayor tiempo de ejecución y mejor AST score.

9. Conclusiones y reflexión

9.1. Conclusiones

El presente trabajo ha evaluado de forma sistemática y controlada el rendimiento de cuatro LLMs locales de acceso abierto: llama3.2:3b, llama3.1:8b, qwen2.5-coder:7b y mistral:7b en las tareas de ofuscación y desofuscación de código JavaScript, comparando sus resultados con la ofuscación tradicional como línea de base. A partir del análisis de los 96 experimentos realizados y de las métricas recogidas en el capítulo anterior, es posible extraer un conjunto de conclusiones estructuradas en torno a los hallazgos más relevantes.

1. La ofuscación tradicional es el único método fiable en las condiciones del experimento

La ofuscación tradicional mediante JavaScript Obfuscator constituye el único método que ofrece garantías reales de funcionalidad y efectividad en el contexto evaluado. Durante la experimentación el método tradicional no produce ningún error de ejecución y alcanza un funcional match del 83.3%.

Su node ratio de 6.17 refleja una transformación estructural profunda, muy superior a la de cualquier LLM evaluado. Ninguno de los modelos se aproxima a estos resultados.

2. Los LLMs no son capaces de ofuscar código de forma fiable

Los resultados de la ofuscación mediante LLM muestran que, salvo en el caso de qwen2.5-coder:7b, los modelos evaluados no consiguen transformar el código de forma funcional y efectiva con una frecuencia aceptable. Los 3 modelos restantes producen errores de ejecución en más de la mitad de los casos: llama3.2:3b falla en 8 de 12 (66.7%), mistral:7b en 10 de 12 (83.3%) y llama3.1:8b en 5 de 12 (41.7%).

En cuanto a la similitud estructural, el AST score tampoco aporta una imagen favorable: llama3.2:3b (0.41) y mistral:7b (0.32) presentan los valores más bajos, lo que significa que en más de la mitad de los casos el código generado difiere estructuralmente de forma significativa respecto al original.

Qwen2.5-coder:7b es la única excepción con un AST score de 0.98 (prácticamente idéntico estructuralmente), si bien su node ratio de 2.33 indica que la expansión del código es modesta en comparación con el método tradicional (6.17), lo que revela que la ofuscación generada es superficial. En suma, los LLMs pueden preservar la sintaxis del código, pero no logran transformarlo de manera profunda ni funcionalmente robusta.

3. La desofuscación es la tarea más difícil para los modelos

Si la ofuscación ya presenta resultados limitados, la desofuscación resulta ser el eslabón más débil de los procesos evaluados. En esta tarea, los modelos deben recuperar la estructura y la lógica de un código que ha sido deliberadamente distorsionado, lo que exige un razonamiento semántico más profundo.

Los resultados muestran que incluso el mejor modelo en ofuscación, qwen2.5-coder:7b, con un 58.3% de functional match, cae hasta el 16.7% en la desofuscación. El caso más extremo es el de llama3.2:3b, que no produce ningún resultado funcionalmente correcto en desofuscación.

El AST score en desofuscación resulta nuevamente poco informativo: modelos con valores estructuralmente altos, como llama3.1:8b (0.81) o qwen2.5-coder:7b (0.91), no se corresponden con un functional match equivalente, lo que indica que los errores son de naturaleza semántica y no puramente formal.

4. El few-shot no mejora de forma uniforme: depende de la tarea

El impacto de la estrategia de prompting varía según la tarea y no sigue un patrón uniforme. En la ofuscación, el few-shot empeora los resultados respecto al zero-shot: el error rate sube del 45.8% al 54.2% y el functional match baja del 33.3% al 16.7% (valores agregados sobre los 4 modelos y 6 scripts, n=48 por estrategia).

Una hipótesis plausible es que los ejemplos del few-shot actúan como un sesgo que lleva al modelo a imitar la superficie del ejemplo en lugar de aplicar una transformación genuina sobre el script objetivo.

En la desofuscación, en cambio, el few-shot produce una mejora notable: el error rate se reduce del 79.2% al 29.2% y el functional match sube del 4.2% al 25.0%.

Esta asimetría sugiere que el few-shot es beneficioso cuando la tarea tiene una naturaleza de recuperación bien definida, pero puede resultar contraproducente cuando requiere creatividad transformadora, como en la ofuscación.

5. La especialización en código es más determinante que el tamaño del modelo

En el contexto de la ofuscación, qwen2.5-coder:7b, el único modelo especializado en código entre los evaluados, obtiene un functional match del 58.3% y un error rate del 8.3%, superando ampliamente a modelos de propósito general de mayor o similar número de parámetros, como llama3.1:8b (8B, 16.7% functional match) o mistral:7b (7B, 8.3% functional match). Con 7B de parámetros, qwen multiplica por 3.5 el rendimiento funcional de llama3.1:8b en esta tarea.

Este resultado sugiere que el ajuste específico para tareas de código tiene un impacto mayor sobre el rendimiento en estas tareas que el incremento en el número de parámetros dentro del rango evaluado. No obstante, esta conclusión debe interpretarse con cautela, ya que el número de modelos evaluados es reducido y no permite generalizaciones amplias.

6. El coste computacional hace inviable el uso de LLMs locales para ofuscación y desofuscación en escenarios reales

La diferencia de tiempos de ejecución entre los LLMs y el método tradicional es de varios órdenes de magnitud. El método determinista opera entre 1.1 y 1.6 segundos por script, mientras que el modelo más rápido llama3.2:3b, requiere una media de 333 segundos en la ofuscación y 546 en la desofuscación. El modelo más lento, mistral:7b supera los 1.000 segundos en ambas tareas. En términos relativos, la ofuscación mediante LLM es al menos 200 veces más lenta que la tradicional.

Esta diferencia, combinada con los resultados de funcionalidad significativamente inferiores, hace que el uso de LLMs locales de tamaño reducido para estas tareas no sea viable en escenarios de producción ni en contextos que requieran un alto volumen de transformaciones. En cambio, su uso podría tener sentido en contextos exploratorios o de análisis puntual, especialmente si se dispone de hardware más potente o de modelos especializados de mayor tamaño.

9.2. Líneas futuras

Por un lado, los modelos evaluados se limitan a aquellos ejecutables en hardware de uso personal con un máximo de 8B de parámetros, dejando fuera modelos más grandes que podrían ofrecer un rendimiento significativamente diferente. Por otro, únicamente se han evaluado dos estrategias de prompting, sin explorar técnicas más avanzadas como el chain-of-thought o el uso de agentes.

Como líneas de trabajo futuras, resultaría de interés ampliar el dataset con scripts de mayor complejidad y diversidad, así como evaluar modelos de mayor tamaño para estas tareas de transformación de código.

La exploración de estrategias de prompting más sofisticadas podrían también contribuir a mejorar la preservación semántica del código generado.

Por último, la extensión del framework a otros lenguajes de programación más allá de JavaScript permitiría comprobar si los patrones de comportamiento observados se mantienen en contextos lingüísticos distintos.

9.3. Reflexión final

Los resultados de este trabajo permiten responder de forma fundamentada a la pregunta central que lo motivó: los LLMs locales de tamaño reducido no son, en las condiciones actuales, una alternativa fiable a los métodos tradicionales para la ofuscación y desofuscación de código JavaScript. Su rendimiento funcional es significativamente inferior, su coste computacional es varios órdenes de magnitud mayor y su comportamiento es menos predecible. Sin embargo, el trabajo identifica condiciones bajo las cuales los modelos muestran un potencial real, especialmente los especializados en código en la tarea de ofuscación y señala el few-shot como una estrategia que puede reducir los errores de ejecución en la desofuscación.

En definitiva, los LLMs representan hoy una herramienta complementaria y exploratoria en este dominio, no un sustituto de los métodos deterministas. Su utilidad real en tareas de ofuscación y desofuscación estará condicionada al avance en el tamaño y la especialización de los modelos, así como al desarrollo de estrategias de prompting más sofisticadas. Este trabajo aporta evidencia empírica reproducible que puede servir de base para investigaciones futuras en ese camino.

10. Bibliografía

10.1. Literatura académica

ⁱ Raitsis, T., Elgazari, Y., Toibin, G. E., Lurie, Y., Mark, S., & Margalit, O. (2025). Code Obfuscation: A Comprehensive Approach to Detection, Classification, and Ethical Challenges. *Algorithms*, 18(2), 54. <https://doi.org/10.3390/a18020054>

ⁱⁱ Brezinski, M., & Ferens, K. (2023). A survey of code obfuscation techniques. *Security and Communication Networks*. <https://doi.org/10.1155/2023/8227751>

ⁱⁱⁱ Song, Y., Lothritz, C., Tang, D., Bissyandé, T. F., & Klein, J. (2024). Revisiting Code Similarity Evaluation with Abstract Syntax Tree Edit Distance. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024, Short Papers)*, pp. 38–46. [10.18653/v1/2024.acl-short.3](https://doi.org/10.18653/v1/2024.acl-short.3)

^{iv} Shao, M., Basit, A., Karri, R., & Shafique, M. (2024). Survey of Different Large Language Model Architectures: Trends, Benchmarks, and Challenges. *IEEE Access*, 12, 128664–128685. [10.1109/ACCESS.2024.3482107](https://doi.org/10.1109/ACCESS.2024.3482107)

^v Zhao, W. X., et al. (2023). A Survey of Large Language Models. *arXiv preprint arXiv:2303.18223*. [10.48550/arXiv.2303.18223](https://doi.org/10.48550/arXiv.2303.18223)

^{vi} Hou, X., et al. (2024). Large Language Models for Cyber Security: A Systematic Literature Review. *arXiv preprint arXiv:2405.04760*. [10.48550/arXiv.2405.04760](https://doi.org/10.48550/arXiv.2405.04760)

^{vii} Li, Y. (2023). A Practical Survey on Zero-shot Prompt Design for In-context Learning. *arXiv preprint arXiv:2309.13205*. [10.48550/arXiv.2309.13205](https://doi.org/10.48550/arXiv.2309.13205)

^{viii} Sahoo, P., et al. (2024). A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. *arXiv preprint arXiv:2402.07927*. [10.48550/arXiv.2402.07927](https://doi.org/10.48550/arXiv.2402.07927)

^{ix} Laneve, C., Spanò, A., Ressi, D., Rossi, S., & Bugliesi, M. (2025). Assessing Code Understanding in LLMs. *arXiv preprint arXiv:2504.00065*. <https://doi.org/10.48550/arXiv.2504.00065>

^x Nikiema, S. L., Samhi, J., Kaboré, A. K., Klein, J., & Bissyandé, T. F. (2025). The Code Barrier: What LLMs Actually Understand?. *arXiv preprint arXiv:2504.10557*. [10.48550/arXiv.2504.10557](https://doi.org/10.48550/arXiv.2504.10557)

^{xi} Beste, D., Menguy, G., Hajipour, H., Fritz, M., Cinà, A. E., Bardin, S., Holz, T., Eisenhofer, T., & Schönherr, L. (2025). Exploring the Potential of LLMs for Code Deobfuscation. In M. Egele et al. (Eds.), *Detection of Intrusions and Malware, and Vulnerability Assessment. DIMVA 2025. Lecture Notes in Computer Science*, vol. 15747, pp. 267–286. Springer, Cham. [10.1007/978-3-031-97620-9_15](https://doi.org/10.1007/978-3-031-97620-9_15)

^{xii} Patsakis, C., et al. (2024). Assessing LLMs in Malicious Code Deobfuscation of Real-World Malware Campaigns. *Expert Systems with Applications*. <https://doi.org/10.1016/j.eswa.2024.124912>

^{xiii} Chen, G., Jin, X., & Lin, Z. (2025). JsDeObsBench: Measuring and Benchmarking LLMs for JavaScript Deobfuscation. *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*. <https://doi.org/10.1145/3719027.3744871>

^{xiv} Mohseni, S., Mohammadi, S., Tilwani, D., Saxena, Y., Ndawula, G. K., Vema, S., Raff, E., & Gaur, M. (2025). Can LLMs Obfuscate Code? A Systematic Analysis of Large Language Models into Assembly Code Obfuscation. *Proceedings of the 39th AAAI Conference on Artificial Intelligence*. <https://doi.org/10.1609/aaai.v39i23.34672>

^{xv} Akil, M. A., Li, A. S., Karim, I., Iyengar, A., Bhattacharya, P., Wang, W., & Bertino, E. (2025). LLMalMorph: On the Feasibility of Generating Variant Malware using Large Language Models. *arXiv preprint arXiv:2507.09411*. [10.48550/arXiv.2507.09411](https://arxiv.org/abs/2507.09411)

10.2. Documentación técnica

Ollama (2025). Ollama — Run large language models locally (v0.13.1).

Recuperado de <https://ollama.com>

(accedido: mayo 2025)

LangChain AI (2025). LangChain — langchain-ollama v1.0.1, langchain-core v1.2.5.

Recuperado de <https://python.langchain.com/docs/integrations/llms/ollama/>

(accedido: mayo 2025)

javascript-obfuscator (2025). JavaScript Obfuscator Tool (v4.0.2).

Recuperado de <https://github.com/javascript-obfuscator/javascript-obfuscator>

(accedido: mayo 2025)

Ariya Hidayat & contributors (2019). Esprima — ECMAScript parsing infrastructure for multipurpose analysis (v4.0.1).

Recuperado de <https://esprima.org>

(accedido: mayo 2025)